

MediFlow Offline Sync Contract

Version: 1.0
Status: Approved design (implementation pending)
Audience: Engineering partners, Apex ops, clinic IT
Last updated: 2026-08-28

This document defines how **offline clinics** run MediFlow on a local server and how data moves between the **clinic node** and the **cloud**. Online-only tenants are unchanged: they always use the cloud API.

1. Goals

Goal	Description
Work without internet	Clinic staff can register patients, collect payment, run queues, chart, lab, and pharmacy during the day on unreliable connectivity.
No silent data loss	Sync is explicit, auditable, and retryable. Failed steps block “complete” status.
Cloud remains SaaS control plane	Billing, module flags, suspend/ban/delete, and payment approval stay on Apex cloud.
One writable node per branch	Avoid multi-writer conflicts by using one reception server per branch TIN.

2. Glossary

Term	Meaning
Cloud	Hosted MediFlow backend + MySQL used for SaaS, billing, and Apex admin.
Clinic node	Reception PC running local backend + local MySQL for one branch TIN .
LAN	Private clinic network; fixed IP for the clinic node (e.g. 192.168.10.10).
Sticky local	While <code>ops_mode = offline</code> , all clinical work uses the clinic node until rules below say otherwise.
Sync	A manager-initiated job: push local → cloud, then pull cloud → local.
Natural key	Business identifier scoped by <code>clinic_tin</code> (e.g. MRN, encounter number), not database integer <code>id</code> .
Days until due	Days until free trial ends or quarterly subscription ends (whichever applies).
Sync freshness	Whether the clinic has synced within the allowed window (see §8).

3. Operating modes (`ops_mode`)

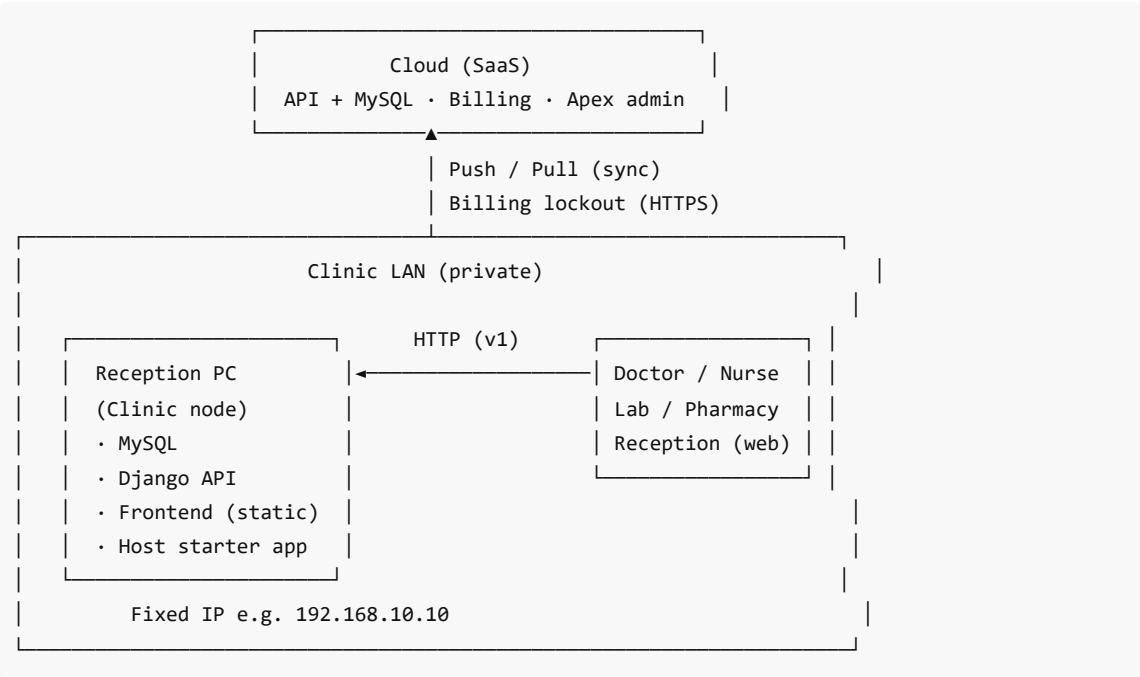
Each tenant has `ops_mode` :

Mode	Daytime behavior
online	All roles use cloud API only. No local clinic node for clinical writes.
offline	Clinical operations use the clinic node on the LAN. Cloud is updated via sync.

Rules:

- A clinic never runs **both** cloud and local clinical writers at the same time.
- **No mid-day mode flip.** Even if Apex approves `ops_mode` change (e.g. offline → online), the new mode takes effect **only after the next successful sync** (push + pull).
- Mode change requires **manager request** in MediFlow and **Apex approval** in `mediflow_admin` (separate **Ops mode requests** page, not mixed with payment approvals).

4. Clinic topology



- **One reception server per branch TIN** (multi-branch orgs = one node per branch).
- Staff browsers open **one URL** on the LAN (see §5).
- **v1 LAN security:** HTTP on an isolated clinical network (see §14). Billing during lockout always uses **cloud over HTTPS**.

5. Clinic host (reception PC)

5.1 Stack

Component	Choice
Database	MySQL (same family as cloud)
API	Django (same codebase as cloud, clinic deployment profile)

Frontend	Option B: production Next.js build served by the backend (single URL, single port)
Launch	Host starter: start backend → health-check → open default browser to frontend

5.2 URLs

Who	URL
Reception (on server)	http://127.0.0.1/ or LAN IP
Other roles	http://<FIXED_LAN_IP>/

Staff do **not** need separate :3000 and :8000 ports in production.

5.3 What the host starter does

1. Ensure MySQL service is running.
2. Start Django (serves /api/* and static frontend).
3. Wait until API health endpoint responds.
4. Open system default browser to the clinic URL.
5. (Recommended) Show tray icon: **Running / Stopped / Last sync / Backup status**.

6. Source of truth

Phase	Clinical data	Billing / Apex control
Clinic day (offline tenant)	Local MySQL on clinic node	Cloud (read on pull; write for lockout payments)
After successful sync	Cloud copy updated; local refreshed from cloud	Cloud
Online tenant	Cloud always	Cloud

6.1 What Apex may change on cloud during the clinic day

- Payment approval / rejection
- Subscription and trial status
- Module flags
- **Suspend / ban / delete** tenant
- Approved **ops_mode** change (effective after next sync)

Local clinic sees these on **pull**, not instantly.

6.2 What must not be overwritten blindly

- **Payments** and **refunds**: append-only events; never delete or rewrite history.
- **Stock**: apply **dispense deltas**, not "set on_hand = N" from a full replace.
- **Catalog**: cloud catalog wins on pull; local catalog edits sync up on push via upsert.

7. Natural keys (sync identity)

Sync APIs use **TIN-scoped natural keys**, not raw integer primary keys.

Entity	Natural key (examples)
Tenant scope	clinic_tin (+ branch_name when multi-branch is explicit)
Patient	(clinic_tin, mrn)
Encounter	(clinic_tin, encounter_number) — field number in DB
Billable service	(clinic_tin, code)
Department	(clinic_tin, name)
Medicine / SKU	(clinic_tin, sku) or agreed code field
Staff user	(clinic_tin, username)
Payment	(clinic_tin, receipt_number) or stable UUID (to be assigned)
Clinical order	(clinic_tin, encounter_number, order_ref) — order_ref to be added if missing
Tenant payment submission	Normal cloud model; synced as normal TenantPaymentSubmission

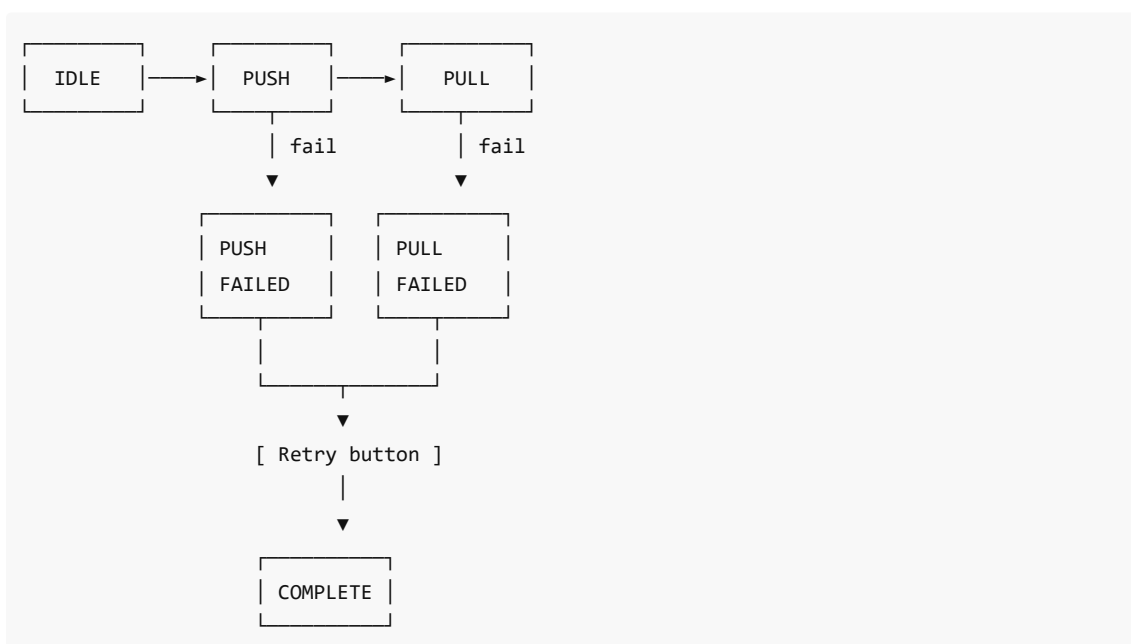
Rule: New records created offline must generate **stable IDs** (UUID or deterministic refs) before push so retries do not duplicate rows.

8. Sync protocol

8.1 Trigger

- **Manual:** Manager clicks **Sync now** in manager portal.
- **Reminder:** If last night's sync was missed → **morning reminder** on manager overview.
- **Mandatory:** After PC outage or overdue freshness → staff login blocked until manager completes sync (manager may access Sync + Billing only).

8.2 Steps (strict order)



Step	Action
1. Push	Upsert local tenant data to cloud (merge by natural keys).
2. Pull	Only if push succeeded. Download cloud state for tenant into local MySQL.
3. Complete	Mark <code>last_successful_sync_at</code> . Apply password/role and ops_mode changes. Unblock staff login if freshness OK.

If pull fails: sync is **incomplete**. Show **Retry** (retry pull from last good push, or full push+pull per implementation). Do not treat the day as synced.

If push fails: do **not** start pull.

8.3 Merge strategy (option B — upsert)

- Match rows by natural keys.
- Update if cloud row is older or field-level rules allow (e.g. encounter status forward-only).
- Insert if missing on cloud.
- Conflicts → `sync_conflict` log for manager review (no silent merge).

8.4 Sync freshness (login gate)

Let `D` = **days until due** (trial end or subscription end).

Condition	Max time without successful sync
<code>D > 7</code>	7 calendar days
<code>D ≤ 7</code>	2 calendar days

If exceeded:

- **Staff roles:** login blocked.
- **Manager:** may open **Sync** and **Billing** (if payment lockout) only until sync completes.

After **PC failure / restore:** first action when node is back = **sync** (same blocking rules).

8.5 Initial go-live (seed)

Before first offline clinic day:

1. Provision clinic node (MySQL + backend + frontend).
2. **Full pull** from cloud: catalog, staff (hashed passwords), tenant settings, empty or existing patients per policy.
3. Manager confirms **Seed complete**, then staff may work offline.

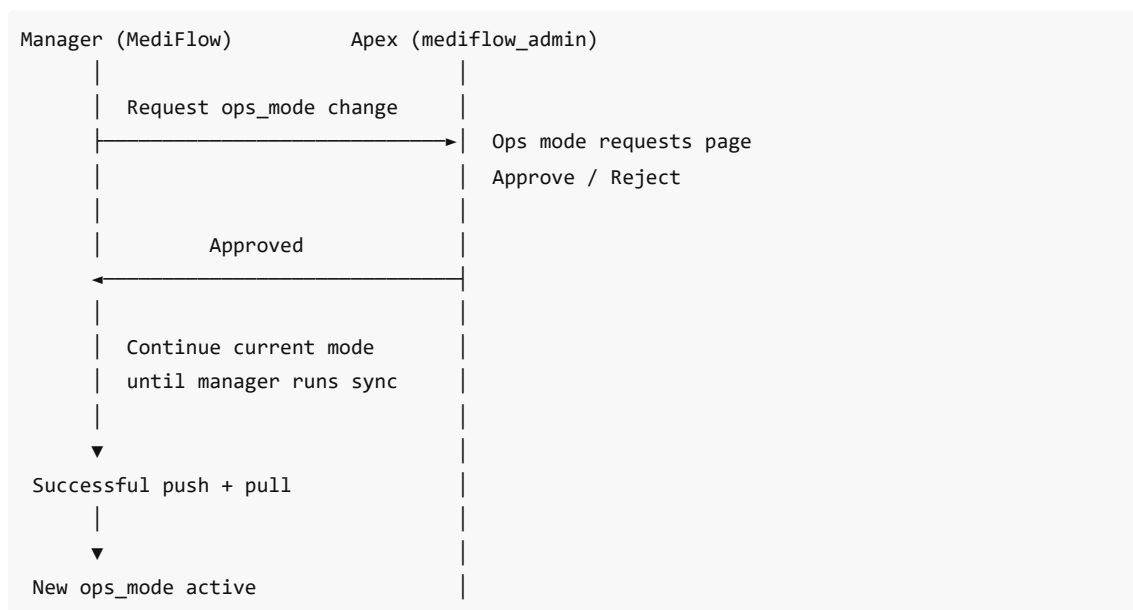
9. Billing vs offline

Situation	Behavior
Warning window (trial ≤ 5 days, quarterly ≤ 10 days)	Clinical offline OK; billing icons in manager UI; payment proof may be stored locally and pushed on next sync.
Before hard due date	Same; local <code>TenantPaymentSubmission</code> rows sync on push as normal submissions.

Hard payment lockdown (trial expired, grace ended, etc.)	Payment proof and approval checks hit cloud only (HTTPS). Manager payment portal on cloud until Apex approves.
Staff during lockdown	Blocked (existing <code>resolve_login_access</code> rules).

Passwords on clinic node: **hashed** (Django-compatible), not decryptable. After pull, credential changes from cloud apply **before next login**.

10. Ops mode change workflow



- **Offline → online:** Final sync must succeed so cloud has all local data before switching daily use to cloud.
- **Online → offline:** After approval, initial **seed pull** to new node before enabling sticky local.

11. Backup strategy (recommended)

Sync is not backup. A failed or delayed sync must not be the only copy of clinic data.

11.1 Tier 1 — Required for every offline clinic

Item	Recommendation
Automated local dump	MySQL <code>mysqldump</code> (or Percona backup) on a schedule: at least daily , ideally every 4–6 hours during clinic hours.
Secondary media	Store dumps on USB drive or second disk on reception PC (rotate 7 daily files).
Pre-sync backup	Automatic dump immediately before each manager-initiated sync.
Manager UI	SHOW: Last backup, Last successful sync, Backup OK / Missing.

11.2 Tier 2 — Strongly recommended

Item	Recommendation
------	----------------

Off-site when online	If internet is briefly available, upload encrypted backup to cloud object storage (S3-compatible) without replacing sync — backup only.
Restore drill	Documented Clinic restore kit : new PC → install host → restore latest dump → manager forced sync.
Backup encryption	Encrypt dumps at rest (password or clinic-specific key stored with manager).

11.3 Tier 3 — Optional later

Item	Recommendation
LAN hot spare	Second low-cost PC with nightly replica (MySQL replication) — failover if reception disk dies.
UPS	Battery backup on reception PC + router so sync/backup can finish during short outages.

11.4 Failure scenarios

Scenario	Expected behavior
Disk dies before sync	Restore from latest backup (Tier 1). Unsynced work since last backup may be lost — UI must show last backup time honestly.
Disk dies, no backup	Data since last successful sync may be unrecoverable . Product + ops must not promise otherwise.
Corrupt local DB	Restore dump; manager sync; Apex support if cloud/local diverged.
Push OK, pull fail	Cloud has new data; local stale. Retry pull mandatory; block staff until complete.

12. Security (v1)

Topic	Decision
LAN transport	HTTP on private clinical VLAN (v1)
Cloud billing	HTTPS always
Wi-Fi	Dedicated clinic network; no guest Wi-Fi on same VLAN as clinical devices
Future	HTTPS on LAN with internal hostname + trusted cert (v1.5)

13. Sync API outline (implementation reference)

Endpoints to implement on **cloud** (clinic node calls when online):

Endpoint	Purpose
POST /api/sync/push/	Body: tenant payload keyed by natural keys; returns push receipt + conflicts.

POST /api/sync/pull/	Returns full tenant snapshot or delta since last_sync_token.
GET /api/sync/status/	Last sync time, freshness, pending conflicts.

Clinic node stores locally:

- sync_runs (id, started_at, push_status, pull_status, error_message)
- last_successful_sync_at
- sync_conflicts (optional queue for manager)

Exact JSON schemas are a follow-up technical appendix.

14. Manager UI requirements

Feature	Required
Sync now button	Yes
Last successful sync timestamp	Yes
Push / pull step status + Retry	Yes
Morning reminder if sync missed	Yes
Sync freshness warning (7d / 2d rules)	Yes
Backup status (last dump, OK/fail)	Yes
Request ops mode change	Yes
Block staff nav when sync overdue	Yes (manager: Sync + Billing only)

15. Apex admin (mediflow_admin) requirements

Feature	Page
Setup / quarterly payment approval	Existing flow
Ops mode change approval	Separate Ops mode requests page
Suspend / ban / delete tenant	Existing; clinic sees on pull

16. Out of scope (v1)

- Real-time sync while clinic is open
- Multiple writable clinic nodes per branch
- Mid-day switch to cloud without sync
- Automatic nightly sync without manager action (reminder only in v1)
- HTTPS on LAN (deferred to v1.5)
- Full conflict-resolution UI beyond list + manual fix

17. Implementation phases (suggested)

Phase	Deliverable
0	Natural keys + UUIDs on new entities; sync_runs tables
1	Cloud push/pull APIs + manager Sync UI
2	Login freshness gates + billing lockout cloud path
3	Clinic host installer (MySQL + Django + static frontend + browser launch)
4	Backup automation + manager backup status
5	Ops mode request/approval in MediFlow + mediflow_admin
6	Conflict list + restore kit documentation

18. Document history

Version	Date	Notes
1.0	2026-08-28	Initial contract from product/architecture review

19. Sign-off (optional)

Role	Name	Date
Product / founder		
Engineering partner		
Apex ops		

For questions or changes to this contract, update the version table and notify all signatories.